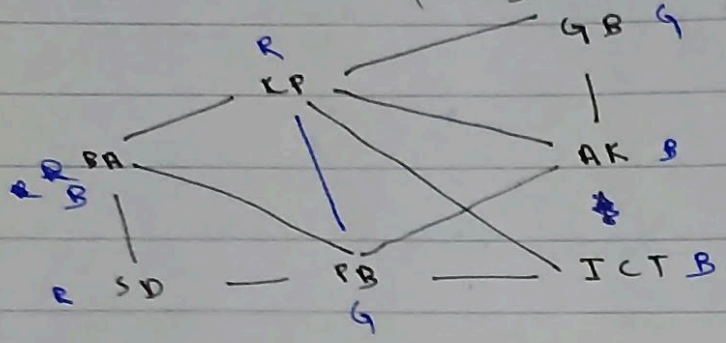
Task 1

24K – 2500

BDS – 5A

Part A: (CSP)



D = { Red, Green, Blue }

GB ≠ KP like all ICT ≠ AK

	GB	KP	AK	ICT	PB	SD	BA
Initial	R,G,B	R,G,B	R,G B	=	=	=	=
GB={R}	R	{G,B}	{G,B}	{R,G,B}	{R,G, B}	{R, G,B}	{R,G,B}
KP={G}	R	G	{B, G}	{R,G, B}	R,G, B	R,G B	R,G,B
AK=B	R	G	B	R,G	R,G, B	R,G B	R,G,B
ICT= R	R	G	B	R	R,G B	=	=

Pearl earl.

Red is already picked by ICT

Green is already used by KP.

Blue is already used by AK.

Go back to ICT and change its clr from Red to Blue.

	GB	KP	AK	ICT	PB	SD	BA
	R	G	B	B	=	=	=
PB=R	R	G	B	B	R	=	=
SD= B	R	G	B	B	R	G	B
	R	G	B	B	R	G	B

B — Dead end here

So back track for SD choose SP = G

so, done all

GB= R, KP= G, AK= B, ICT = B

PB= R SD= G BA= B

2) By R∧R∨

GB	KP	AK	ICT	PB	SD	BA
R	G	✓= {B}	= {R,G,B}	= {R,G,B}	= {R,G,B}	= {R,B} ✓
R	G	B	R,G,B	R,G,B	R,G,B	R,B
R	G	B	R,G,B	G,R	G,R	B
R	G	B	R,G,B	R	G	B
R	G	B	B	R	G	B

Degree Heuristic

(choose one that is highest degree.)

AK = 3 ✓ PB = 5 ✓

KP = 5 ✓ SP = 2 ✓ BA = 3 ✓

GB	AK	KP	ICT	PB	SD	BA
=	=	R	=	=	=	=
=	=	R	=	G	=	=
=	=	R	=	G	=	B
=	B	R	=	G	=	B
G	B	R	B	G	R	B

c)

$P(Re, At, Bu, OV, Cr, AL, Co, Es) =$

$P(Re)\, P(At) \times P(Bu|Re)\, P(OV|At) \times P(Cr|Bu, O) \times P(AL|Cr, At) \times P(Co|Cr) \times P(Es|AL, CO).$

a) Find probability that server crashes.

$$P(Cr) = \sum_{Bu, OV} P(Cr|Bu, OV)\, P(Bu)\, P(OV)$$

$P(Bu) = P(Bu|Re)\, P(Re) + P(Bu|\neg Re)\, P(\neg Re)$

$P(Bu) = 0.3 \times 0.2 + 0.07 \times 0.8$

$P(Bu) = 0.10 \qquad P(\neg Bu) = 0.90.$

$P(OV) = P(OV|At)\, P(At) + P(OV|\neg At)\, P(\neg At)$

$\Rightarrow 0.6 \times 0.1 + 0.1 \times 0.9$

$P(OV) = 0.15$

$P(\neg OV) = 0.85$

PART A: PERFORMANCE ANALYSIS

Performance analysis is used to compare Basic Backtracking and Optimized Backtracking.
The program uses three counters:
1. Assignment Iterations:
It counts how many times the program tries to assign a color to a region.
2. Backtracks:
It counts how many times the program goes back because the selected color does not work.
3. Forward Checks:
It counts how many invalid colors are removed from the domains of neighboring regions.
BASIC BACKTRACKING
Basic Backtracking assigns colors using a fixed random order. It checks regions one by one and does not use Forward Checking.

OPTIMIZED BACKTRACKING

Optimized Backtracking uses three techniques:

1. MRV selects the region with the fewest available colors.
2. Degree selects the region with the most unassigned neighbors when there is a tie.
3. Forward Checking removes invalid colors from neighboring regions after an assignment.

PERFORMANCE COMPARISON

The basic algorithm may require more assignment attempts because it follows a fixed order.

The optimized algorithm can reduce unnecessary attempts because MRV and Degree help select better regions. Forward Checking also detects possible conflicts earlier.

The actual number of assignment iterations, backtracks, and forward checks is taken from the program output.

ILLUSTRATION OF FORWARD CHECKING

Suppose we assign:

KP = Red

The neighboring regions of KP include BA, PB, ICT, GB, and AK.

Initially, the domain of PB is:

PB = {Red, Green, Blue}

Since PB is a neighbor of KP, it cannot use Red.

After Forward Checking:

PB = {Green, Blue}

This helps detect conflicts before assigning colors to all regions.

In this task, we implemented two approaches to solve the map-coloring problem.

Basic Backtracking uses a fixed variable order, while Optimized Backtracking uses MRV, Degree, and Forward Checking.

Both approaches solve the map-coloring problem, but the optimized approach can reduce unnecessary work by selecting variables more carefully and detecting conflicts earlier..

Task 2
HEURISTIC FUNCTION 1: LINEAR MOBILITY DIFFERENCE

The first heuristic evaluates a game state by calculating the difference between the number of legal moves available to AI and Human.

Formula:

H1(S) = M_AI - M_Human

Where:
M_AI = Number of legal moves available to AI.
M_Human = Number of legal moves available to Human.

Example:

Suppose AI has six legal moves and Human has three legal moves.

H1(S) = 6 - 3 = 3

A positive value indicates an advantage for AI, while a negative value indicates an advantage for Human.

Purpose:
This heuristic encourages the AI to maintain more legal moves than its opponent.

HEURISTIC FUNCTION 2: WEIGHTED CELL EVALUATION

The second heuristic evaluates the quality and location of available destination cells.

In this approach, cells closer to the center of the board receive higher weights because they may provide access to more movement options.

Formula:

H2(S) = W_AI - W_Human

Where:

W_P = Sum of Weight(c) for all legal moves of player P.

The weight of a destination cell is calculated as:

Weight(r,c) = 1 + (4 - |r-2| - |c-2|) / 4

This means:

1. Central cells receive higher weights.
2. Corner cells receive lower weights.
3. Players with access to more valuable cells receive a higher evaluation score.

Purpose:
This heuristic encourages the AI to consider positional advantages instead of only counting legal moves.

PYTHON IMPLEMENTATION RESULTS

Both heuristic functions were tested using the same initial game state and a maximum search depth of four plies.

BENCHMARK RESULTS

Heuristic 1:
Evaluation Method: Linear Mobility
Best AI Move: (2, 2)
Evaluation Score: 1
Nodes Explored: 2,888
Branches Pruned: 3,532
Search Depth: 4

Heuristic 2:

Evaluation Method: Weighted Cells
Best AI Move: (3, 3)
Evaluation Score: 2.0
Nodes Explored: 3,324
Branches Pruned: 3,744
Search Depth: 4

PERFORMANCE ANALYSIS

Heuristic Function 1 uses linear mobility difference to evaluate game states.

It explored 2,888 nodes and selected position (2, 2) as its best move.

Heuristic Function 2 uses weighted cell evaluation. It explored 3,324 nodes and selected position (3, 3) as its best move.

The second heuristic explored 436 more nodes than the first heuristic.

This indicates that the weighted evaluation method involved more search exploration in this experiment.

Although Heuristic 2 produced a higher numerical evaluation score, the scores are calculated using different evaluation functions. Therefore, they cannot be directly compared as measures of playing strength.
COMPARISON OF BOTH HEURISTICS

Heuristic 1:
Evaluation Method: Mobility Difference
Main Focus: Number of legal moves
Complexity: Low
Positional Information: Limited
Nodes Explored: 2,888

Heuristic 2:
Evaluation Method: Weighted Cell Values
Main Focus: Quality of available cells
Complexity: Moderate
Positional Information: Greater
Nodes Explored: 3,324

The first heuristic is computationally simpler, while the second provides additional positional information.

Both heuristics successfully generated AI moves using Alpha-Beta Pruning.

CONCLUSION OF PART B

In Part B, an AI agent was developed to play the Isolation game on a 5 × 5 board.

The Minimax algorithm was implemented with Alpha-Beta Pruning to make decisions against a Human opponent.

The maximum search depth was limited to four plies to control computational complexity.

Two heuristic evaluation functions were designed.

The first heuristic calculates the difference between the number of legal moves available to AI and Human.

The second heuristic evaluates available cells using positional weights, giving greater importance to central locations.

The benchmark results show that the first heuristic explored fewer nodes, while the second considered additional positional information.

Overall, this implementation demonstrates how adversarial search, heuristic evaluation, and pruning techniques can be combined to develop an intelligent game-playing agent.
Task3

AK = 3 ✓ PB = 5 ✓

KP = 5 ✓ SP = 2 ✓ BA = 3 ✓

GB	AK	KP	ICT	PB	SD	BA
=	=	R	=	=	=	=
=	=	R	=	G	=	=
=	=	R	=	G	=	B
=	B	R	=	G	=	B
G	B	R	B	G	R	B

c)

$P(Re, At, Bu, OV, Cr, AL, Co, Es) =$

$P(Re)\, P(At) \times P(Bu|Re)\, P(OV|At) \times P(Cr|Bu, O) \times P(AL|Cr, At) \times P(Co|Cr) \times P(Es|AL, CO).$

a) Find probability that server crashes.

$$P(Cr) = \sum_{Bu, OV} P(Cr|Bu, OV)\, P(Bu)\, P(OV)$$

$P(Bu) = P(Bu|Re)\, P(Re) + P(Bu|\neg Re)\, P(\neg Re)$

$P(Bu) = 0.3 \times 0.2 + 0.07 \times 0.8$

$P(Bu) = 0.10 \qquad P(\neg Bu) = 0.90.$

$P(OV) = P(OV|At)\, P(At) + P(OV|\neg At)\, P(\neg At)$

$\Rightarrow 0.6 \times 0.1 + 0.1 \times 0.9$

$P(OV) = 0.15$

$P(\neg OV) = 0.85$

	Overload	Calculation	Probability
B4	0V	0.95 × 0.10 × 0.10	0.014
B4	10V	0.10 × 0.85 → 0.60	0.051
¬B4	0V	0.90 × 0.15 × 0.40	0.054
¬B4	10V	0.90 × 0.85 × 0.03	0.022

$$P(Cr) = 0.014 + 0.051 + 0.054 + 0.022$$

$$\boxed{P(Cr) = 0.1422}$$

b) $$P(AL) = \sum_{Cr, Dt} P(AL \mid Cr, At)\, P(Cr, Dt)$$

$P(Cr \mid At) = 0.30$ $P(Cr \mid \neg At) = 0.12$

$P(Dt) = 0.1$

Cr	At	P(Cr, Dt)	P(OL | Cr, At)	Product
Cr	At	0.030	0.99	0.030
Cr	¬At	0.111	0.90	0.100
¬Cr	At	0.069	0.70	0.048
¬Cr	¬At	0.788	0.05	0.039

$$\boxed{P(OL) = 0.2186}$$

c) $$P(Cr \mid Re) = \frac{P(Cr, Re)}{P(Re)}$$

$$P(Cr \mid Re) = 0.042 + 0.153 + 0.017$$

d) $P(E_3) = \sum_{AL,CO} P(E_3 | AL, CO) \cdot P(AL, CO)$.

$P(E_3) = 0.11 \times 0.95 + 0.09 \times 0.60 + 0.08 \times 0.40 + 0.69 \times 0.01$

$\boxed{P(E_3) = 0.2147}$

e) $P(At | AL, TCO) = \frac{P(At, AL, TCO)}{P(AL, TCO)}$

$P(AL, TCO) = 0.0987$

$P(At, AL, TCO) = 0.04817$

$P(At | AL, TCO) = \frac{0.0481}{0.0987}$

$\boxed{P(At | AL, TCO) = 0.4881}$

f) $P(Re | Cr, TAL) = \frac{P(Re, Cr, TAL)}{P(Cr, TAL)}$

$\boxed{P(Re | Cr, TAL) = 0.38053}$